Release bundles (the runtime publish unit)

TBK SPEC 36:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 36:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Why (the two budgets, measured 2026-09-23/24) 4

2. The bundle (wire format) 5

3. The publish shape (bundle-era lines) 6

4. The resolver path (spec 05 §2 amendment) 7

5. The registry mirror (spec 04 §2 amendment) 8

6. The release-topology policy (spec 13 amendment, locked 2026-09-24) 9

7. Named errors (new class; existing classes unchanged) 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: **PLANNED** (locked direction 2026-09-24; ships with tebako-release v0.2.0 + the tebako-resolve bundle fetch path — the v0.16.28 line-shard recovery runs the per-file shape to completion, and pre-bundle releases stay installable forever, spec 13 §8).

Normative specification of the **bundle**: the single distribution artifact that carries one runtime leg's bytes (interpreter exe + env image + platform support files) as one gzipped tar. The bundle replaces the per-file asset enumeration of spec 13 §2a on bundle-era release lines; it changes what a factory PUBLISHES and what a resolver FETCHES, never what the store holds (spec 05 §3's layout is unchanged — the bundle unpacks into it).

1. Why (the two budgets, measured 2026-09-23/24)

- **The 1,000-asset ceiling.** GitHub caps a release object at 1,000 assets (`422 … file_count limited to 1000 assets per release`). The ruby factory's catalog is 23 versions × 7 platforms; at the per-file shape's ~10–14 assets per leg (exe, image, both `.sha256`, the shard, windows DLLs, and an `.asc` of each on signing-enabled lines) a catalog publish is ~1,800 assets — it cannot converge on one release object. The v0.16.28 recovery line-sharded the catalog onto per-line tags (`v0.16.28-ruby4.0`, …) via `TEBAKO_RELEASE_TAG`; that plumbing stays as the migration-time escape hatch (§6).
- **The API budget.** Every asset is REST calls (upload, list, sign, re-list). The same recovery exhausted the tebako-ci identity's 5,000 req/h sliding window mid-fleet (tebako-release#3). The per-leg asset count is the multiplier of BOTH budgets; consumption is always paired (the loader fetches exe and env image together), so the per-leg bundle is the natural unit.

© 2026 The Tebako Project

Budget table (per leg / catalog of 23 × 7):

TABLE 1

shape	unsigned	signing-enabled
per-file (today)	~5 / ~800–1,100	~10–14 / ~1,800
bundle (this spec)	3 / 483	5 / 805

The bundle brings the whole catalog back under the ceiling even on signing-enabled lines; line shards (§6) remain the headroom, not the routine.

2. The bundle (wire format)

`<stem>.tar.gz` where `<stem>` is the package stem (`tebako-runtime-<ver>-<lang>-<lv>-<triplet>` — suffix-less, spec 05 §2's spelling rule and era law, on windows too). A gzipped tar whose members are named by their served spellings, in this fixed order (deterministic bytes — a re-run of identical inputs yields a digest-identical bundle, so the write-once digest-match skip stays cheap):

FIGURE 1

```
<stem>[.exe]        # the interpreter exe (mode 0755)
<stem>.tfs          # the env image (mode 0444 at install)
<stem>.dll …        # windows support DLLs under their
staged basenames,
                    # zero or more, sorted (today: the one
runtime DLL)
SHA256SUMS          # coreutils "<sha>  <file>\n", one line
per member
```

- `SHA256SUMS` is the LAST member and pins every other member; the per-member digests mirror the shard's sha fields (§3) exactly.
- gzip headers carry `mtime=0` and no original-name field. Tar headers carry no uid/gid/uname/gname; modes as above.
- Members are regular files only. No directories, symlinks, device nodes, absolute paths, or `..` segments — a bundle violating the member grammar is `InvalidBundle` (§7), never a best-effort unpack.
- The bundle is built ONCE per leg by the publish invocation from the leg's staged bytes (tebako-release owns the builder — in-process `Gem::Package::TarWriter` + `Zlib::`

© 2026 The Tebako Project

`GzipWriter`; no shell-outs), after the leg's gates pass, from the same bytes the per-file shape would have uploaded.

3. The publish shape (bundle-era lines)

Per leg, the publish invocation writes — write-once, spec 13 §2a's immutability rule unchanged:

FIGURE 2

```
<stem>.tar.gz          # the bundle
<stem>.tar.gz.sha256   # the per-asset sidecar (coreutils
shape)
<stem>.manifest.json   # the per-package shard (the
authority, §2's role)
```

plus, on signing-enabled lines, an `.asc` of EACH of the three (spec 09 §5's every-served-name rule — three signatures where the per-file shape needed eight or more).

The shard keeps today's identity anchors and gains ONE additive key:

FIGURE 3

```
{
  "tebako_version": "0.17.0", "contract_version": 2,
  "ruby_version": "4.0.7", "platform": "macos-arm64",
  "filename": "tebako-runtime-0.17.0-ruby-4.0.7-macos-arm64",
  "sha256": "…", "size_bytes": 38683544,
  "abi": "arm64-darwin-23",
  "image": {"filename": "….tfs", "sha256": "…", "size_bytes":
 7658081},
  "bundle": {"filename": "tebako-runtime-0.17.0-ruby-4.0.7-
macos-arm64.tar.gz",
             "sha256": "…", "size_bytes": 46012377,
             "signature": {"keyid": "efc3c250f7862a48",
"asc": "….tar.gz.asc"}}
}
```

- `filename` stays the EXE's spelling and the per-member sha fields (`sha256`, `image.sha256`, `dll.*`) keep pinning the UNPACKED members — the store verifies members

© 2026 The Tebako Project

against them post-unpack (§4), and old readers keep matching the identity triple exactly as today.

- `bundle` is additive: pre-bundle readers ignore it (the additive rule); bundle-era readers prefer it. A shard declaring `bundle` without the bundle's assets on the release is an invalid publish — the leg fails before it ships, exactly like a `signature` declaration without its `.asc` (spec 09 §4's invalid-signing-state rule extended to the bundle block).
- The windows `dll` metadata block (`install_as` et al.) is unchanged: DLLs ride the bundle as members and install by the shard's declared spellings.

Co-publish (the lazy arm's serving requirement, spec 39 §7). A bundle-era line whose consumers include a lazy-mount loader ALSO serves the per-file assets of spec 13 §2a's enumeration — the bare exe under its historical spelling, `<stem>.tfs`, `<stem>.dll` (windows) — each with its own `.sha256` sidecar, plus an `.asc` of each on signing-enabled lines. The shard's grammar is otherwise unchanged: the per-file blocks already pin exactly these bytes (the member pins ARE the standalone pins — one staged file, one digest, two serving forms); the shard gains ONE additive witness, top-level `"per_file_assets": true` (runtime-manifest MINOR 2), and declares the entry/facet `signature` blocks exactly as the per-file era does (the every-served- name rule covers the standalone set again). A shard carrying the witness without the assets on the release is the same invalid publish as a `bundle` block without its bundle — the leg fails before it ships and the audit names the gap. The bundle stays the ONE fetch of the default eager path (§4) — co-publish changes what the release SERVES, never what the bundle carries or how it verifies. The §4 compat window closes on co-published lines: a pre-bundle reader ignores `bundle` (the additive rule) and takes the per-file path against the same pinned bytes. Asset budget: the union is 9–11 assets per leg unsigned (18–22 on signing-enabled lines); line tips (5 rubies × 7 platforms) stay far under the 1,000-asset ceiling, and catalog republications ride §6's line shards as always. The mode is the factory's declared policy (the release tooling's adapter seam), never an env knob — a factory opts in when its consumers include the lazy arm.

4. The resolver path (spec 05 §2 amendment)

Preference order is unchanged (shard → derived monolith → line index); the era branch happens INSIDE the shard case:

- `bundle` present → **bundle fetch**: download `bundle.filename` + its `.sha256` sidecar (+ `.asc` when `bundle.signature` declares it) → verify the detached signature FIRST when declared (spec 09 §4's runtime-fetch point, always strict: 71 invalid / 72 untrusted) → verify the bundle sha256 (exit 70 on mismatch) → unpack in-process (Rust `tar` + `flate2`; no shell-outs, invariant 1) with the §2 member-grammar validation → verify EVERY member against the shard's per-member pins (`InvalidBundle` on mismatch — a bundle whose members disagree with the signed shard is an attack surface, never a

© 2026 The Tebako Project

tolerance) → write the spec 05 §3 store layout exactly as today (exe 0755, image 0444, the `sha256/origin` markers from the member pins) via tmp + rename — a partial install stays invisible.

- `bundle` absent → the per-file path, forever (keep-forever, spec 13 §8). The era branch is per-release, never global.
- Old resolver, bundle-era release (the compat window): the shard matches, `filename` flows verbatim, and — on a bundle-only line — the exe asset does not exist on the release, so the fetch fails with the named not-found error (exit-69 class) naming the identity triple and hinting that the line requires a bundle-era tebako. Never an exec of wrong bytes, never a silent fallback: old readers break LOUD, which the additive-key design makes the ONLY failure mode. On a CO-PUBLISHED line (§3) the window closes: the per-file assets exist again and the old reader's per-file path installs the same pinned bytes.
- One fetch replaces three or more: the journal records the bundle base and channel exactly as today (spec 05 §2's per-engine chain applies unchanged — the bundle is just the asset the chain supplies).

5. The registry mirror (spec 04 §2 amendment)

A bundle-era runtime entry's platform rows name the BUNDLE: `artifact: <stem>.tar.gz`, `sha256` pins the bundle. The rows stay the resolution mirror and audit pin; the runtime download itself flows the shard (§4). Per-file-era rows keep naming the exe — the registry is a faithful mirror of what the pinned release serves, era by era, and a `status: withdrawn` mark means the same thing on both. A CO-PUBLISHED line's row is unchanged — it still names the bundle (the preferred eager fetch): the row's `blksum` pin (spec 39 §3) is the lazy wire's only registry mirror, and the standalone per-file asset names derive from the stem (spec 05 §2's spelling rule), so the row grammar carries nothing new.

© 2026 The Tebako Project

6. The release-topology policy (spec 13 amendment, locked 2026-09-24)

- Tips by default. A routine factory release publishes the `full` set (line tips: 5 rubies × 7 platforms ≈ 385 per-file assets, ≈ 175 bundle assets) — comfortably under the ceiling on either shape.
- Catalog republication is for migrations only (a format switch like dwarfs→limnifs), and a catalog run DERIVES its shard tags itself: `v<ver>-<engine><line>-<platform>` per (ruby line × platform group) — the platform group's vocabulary is `linux-gnu` / `linux-musl` / `macos` / `windows` (one shard serves both arch legs of its group) — no operator input (the v0.16.28 manual `TEBAKO_RELEASE_TAG` input remains as the override). A per-platform shard tag is the publish unit because concurrent platform legs never share one release object (the ghost-release race); a version's line-level tag MAY also exist (the 0.17.0 shape) but is never required. Consumers derive: a registry's version-level `release.ref` names any one published shard of the version; a reader substitutes its own platform group's segment (`v0.17.1-ruby3.3-windows` read on macOS → `v0.17.1-ruby3.3-macos`), and a line-level pin probes its per-platform shard first, the line tag second, the monolith last. A row carrying the additive per-row `release.ref` (registry schema MINOR 6, tebako#711) names its serving shard tag EXPLICITLY — it wins over the version-level derivation (spec 04 §2). The grammar's ONE code owner is tpkg (`Platform::shard_tag_segment`, `shard_tag_substitute`, `shard_tag_append`); a tag outside the vocabulary is opaque and never rewritten. The AUDIT rides the tail of each (platform × shard) run — the duty is per-platform by construction, and the platform run owns the matrix truth (its own compute outputs; never a recomputation, never a cross-run stale read); the coordinator's release job keeps only the registry renders, one bot PR per shard tag.
- Withdrawal bounds growth (spec 04 §2): a superseded patch version takes `status: withdrawn` at the line's next publish rather than being re-hosted on every migration. The catalog is the historical record; the registry is what resolvers walk.

7. Named errors (new class; existing classes unchanged)

- `InvalidBundle` — member-grammar violation, member-set mismatch vs the shard, member checksum mismatch, or an undecodable tar/gzip stream. Exit 70's integrity class at fetch; the message names the member and the shard it disagrees with. Never a partial store write (tmp + rename), never a skipped member.

© 2026 The Tebako Project

- Bundle fetch/signature failures reuse the spec 09 §4 classes unchanged: 69-class not-found, 70 sha256, 71 invalid/absent signature, 72 untrusted keyid.

© 2026 The Tebako Project